Pipeline Construction, Model Training, and Baseline Evaluation

Week 5, Day 2

Lumumba W. Victor
Trainer — Data Analyst — ML Expert

MediaCrest Training College

July 15, 2026

Abstract

This session focuses on transitioning from exploratory data analysis to building robust machine learning models. We will construct leak-proof preprocessing pipelines, train multiple classification algorithms using advanced hyperparameter tuning, and establish a rigorous baseline performance evaluation.

- **Pipelines:** Robust numerical and categorical preprocessing.
- **Model Selection:** Training 7 diverse classification algorithms.
- **Tuning:** `RandomizedSearchCV` with stratified cross-validation.
- **Evaluation:** Comprehensive metrics on an untouched hold-out test set.

Our ultimate goal is to establish a reliable, high-performing baseline for predicting Atherosclerotic Heart Disease (AHD).

Session Objectives

- Build robust, leak-proof preprocessing pipelines using `ColumnTransformer`.
- Handle numerical and categorical data appropriately to prevent data leakage.
- Train multiple classification algorithms (LR, KNN, SVM, NB, DT, RF, XGBoost).
- Implement hyperparameter tuning using `RandomizedSearchCV`.
- Evaluate baseline models on an untouched hold-out test set using a comprehensive suite of metrics.

Agenda

1. **Building Robust Preprocessing Pipelines** (Numerical, Categorical, ColumnTransformer)
2. **Model Selection & Hyperparameter Tuning Setup** (Algorithms, Grids, RandomizedSearchCV)
3. **Model Training Execution** (Running pipelines, extracting parameters)
4. **Baseline Evaluation on Hold-Out Test Set** (Metrics, Confusion Matrices, Comparison)

Part 1: Building Robust Preprocessing Pipelines

"Ensuring data integrity and preventing leakage."

The Need for Robust Preprocessing

- Machine learning models require strictly numerical input.
- Real-world clinical data contains missing values and mixed data types.
- Preprocessing must be applied consistently to both training and test data.
- **Crucial Rule:** Preprocessing parameters (like mean or median) must be learned *only* from the training data to prevent data leakage.

The Solution

Scikit-Learn `Pipeline` and `ColumnTransformer` encapsulate these steps, ensuring leak-proof transformations.

Numerical Pipeline: Imputation

- Missing values in numerical features can break many algorithms.
- **Strategy:** `SimpleImputer(strategy='median')`

Why Median?

- Robust to outliers compared to the mean.
- Preserves the central tendency of skewed clinical variables (e.g., cholesterol, oldpeak).

The imputer calculates the median strictly on the training set and applies it to both train and test sets.

Numerical Pipeline: Scaling

- Many algorithms (LR, SVM, KNN) are highly sensitive to the scale of features.
- **Strategy:** `StandardScaler`

Transformation

Transforms features by subtracting the mean and dividing by the standard deviation.

$$z = \frac{(x - \mu)}{\sigma}$$

Results in a distribution with mean $= 0$ and standard deviation $= 1$.

Ensures that no single feature dominates the objective function purely due to its magnitude.

Numerical Pipeline Implementation

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Define the numerical pipeline
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

Key Takeaway

The `Pipeline` ensures that the scaler uses the mean and standard deviation calculated *only* from the training folds during cross-validation.

Categorical Pipeline: Imputation

- Missing values in categorical features require a different approach.
- **Strategy:** `SimpleImputer(strategy='most_frequent')`

How it Works

- Replaces missing values with the mode (most frequent category) of that feature.
- Computed strictly on the training data to avoid leaking test set distributions.
- Ideal for nominal variables like `ChestPain` or `Thal`.

Categorical Pipeline: Handling Unknowns

- Real-world test data might contain categories not seen during training.
- **Strategy:** `OrdinalEncoder` with `handle_unknown='use_encoded_value'`

Why is this Important?

- Assigns a specific encoded value (e.g., -1) to unseen categories.
- Prevents the model from crashing during inference on new, real-world patient data.
- Allows the model to treat "unknown" as a distinct, informative state.

Categorical Pipeline Implementation

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder

# Define the categorical pipeline
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(
        handle_unknown='use_encoded_value',
        unknown_value=-1
    ))
])
```

The ColumnTransformer: Combining Pipelines

- Datasets typically contain both numerical and categorical features.
- `ColumnTransformer` allows applying different pipelines to different columns simultaneously.

Benefits

- Ensures that numerical and categorical preprocessing steps remain isolated.
- Automatically aligns the transformed features back together.
- Outputs a single, fully preprocessed numerical array ready for modeling.

ColumnTransformer Implementation

```python
from sklearn.compose import ColumnTransformer

# Assume num_features and cat_features are lists of column names
preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_pipeline, num_features),
        ('cat', cat_pipeline, cat_features)
    ],
    remainder='drop' # Use 'passthrough' to keep unlisted columns
)
```

Part 2: Model Selection & Hyperparameter Tuning Setup

"Choosing the right algorithm and optimizing its configuration."

Overview of Classification Algorithms

We will evaluate a diverse ensemble of algorithms to establish a strong baseline:

- **Linear:** Logistic Regression, Support Vector Machines (SVM)
- **Instance-Based:** K-Nearest Neighbors (KNN)
- **Probabilistic:** Naive Bayes
- **Tree-Based:** Decision Tree, Random Forest, XGBoost

Why this Diversity?

Different algorithms capture different types of relationships (linear, non-linear, interactions) in the data, ensuring we don't miss the optimal predictive pattern.

Linear & Distance-Based Models

- **Logistic Regression:** Baseline linear model. Fast, interpretable, good for linear boundaries.
- **Support Vector Machines (SVM):** Finds the optimal hyperplane. Effective in high-dimensional spaces. Uses kernels for non-linear boundaries.
- **K-Nearest Neighbors (KNN):** Non-parametric. Classifies based on the majority class of the 'k' closest training examples. Highly sensitive to scale and dimensionality.

Probabilistic & Tree-Based Models

- **Naive Bayes:** Probabilistic classifier based on Bayes' theorem. Assumes feature independence. Fast and works well with high-dimensional data.
- **Decision Tree:** Splits data based on feature thresholds. Highly interpretable but prone to overfitting.
- **Random Forest & XGBoost:** Ensemble methods. Random Forest uses bagging; XGBoost uses boosting. State-of-the-art for tabular clinical data.

Introduction to Hyperparameter Tuning

Parameters vs. Hyperparameters

- **Parameters:** Learned from data (e.g., weights in Logistic Regression).
- **Hyperparameters:** Set before training (e.g., learning rate, tree depth, k in KNN).

- **Goal:** Find the optimal hyperparameter combination that maximizes model performance without overfitting.
- **Methods:** Grid Search (exhaustive) vs. Randomized Search (stochastic).

Defining Hyperparameter Grids (Part 1)

```python
# Search spaces for linear and distance-based models
param_grids = {
    'lr': {
        'C': [0.01, 0.1, 1, 10],
        'solver': ['lbfgs', 'liblinear']
    },
    'knn': {
        'n_neighbors': [3, 5, 7, 9],
        'weights': ['uniform', 'distance']
    },
    'svm': {
        'C': [0.1, 1, 10],
        'kernel': ['linear', 'rbf'],
        'gamma': ['scale', 'auto']
    }
}
```

Defining Hyperparameter Grids (Part 2)

```
# Search spaces for tree-based ensemble models
param_grids.update({
    'rf': {
        'n_estimators': [100, 200],
        'max_depth': [None, 5, 10],
        'max_features': ['sqrt', 'log2']
    },
    'xgb': {
        'n_estimators': [100, 200],
        'max_depth': [3, 5, 7],
        'learning_rate': [0.01, 0.1, 0.2]
    }
})
```

Tree-based models require tuning ensemble size, depth, and learning rates to prevent overfitting.

Cross-Validation: 5-Fold Stratified

- **K-Fold CV:** Splits training data into K subsets. Trains on K-1, validates on 1. Repeats K times.
- **Stratified:** Ensures each fold maintains the same proportion of classes as the original dataset.
- **Why 5-Fold?** Balances computational cost with a reliable, low-variance estimate of model performance.

Crucial for Healthcare

Stratification is critical for imbalanced datasets to ensure minority classes (e.g., sick patients) are adequately represented in every validation fold.

RandomizedSearchCV: Concept & Benefits

- Instead of trying every combination (GridSearch), it samples a fixed number of random combinations from the parameter distributions.

Benefits

- **Computationally Efficient:** Especially beneficial when tuning many hyperparameters or using complex models like XGBoost.
- **Effective:** Often finds a near-optimal combination much faster than exhaustive grid search.

Configuration: Optimizing for `roc_auc` (ideal for imbalanced classification) and tracking `fit_time`.

RandomizedSearchCV Implementation

```
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold

search = RandomizedSearchCV(
    estimator=model_pipeline,
    param_distributions=param_grid,
    n_iter=20, # Number of random samples to try
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='roc_auc',
    n_jobs=-1, # Use all CPU cores
    random_state=42
)
```

Part 3: Model Training Execution

"Running the pipelines and extracting insights."

Running the Training Pipelines

1. Iterate through the dictionary of models and their respective hyperparameter grids.
2. For each model, wrap it in a final pipeline with the `preprocessor`.
3. Initialize the `RandomizedSearchCV` object.
4. Fit the search object on the training data (`X_train`, `y_train`).

Execution

This process is computationally intensive but fully automated, parallelized (`n_jobs=-1`), and strictly leak-proof.

Extracting Best Hyperparameters

```python
# After fitting the search object on the training data
best_model = search.best_estimator_
best_params = search.best_params_
best_score = search.best_score_

print(f"--- Results for {model_name} ---")
print(f"Best Parameters: {best_params}")
print(f"Best Cross-Validation ROC-AUC: {best_score:.4f}")
```

These attributes provide the optimal configuration found during the randomized search.

Analyzing Cross-Validation Scores

- `search.cv_results_` provides a detailed dictionary of all evaluated combinations.

What to Analyze

- **Mean Test Scores:** The average performance across the 5 folds.
- **Standard Deviation:** Indicates model stability. High variance suggests sensitivity to data splits.
- **Mean Fit Time:** Identifies computationally expensive models, crucial for real-time clinical deployment.

This helps in selecting a model that balances performance, stability, and inference speed.

Part 4: Baseline Evaluation on Hold-Out Test Set

"The ultimate test of generalization."

The Hold-Out Test Set

- The test set (`X_test`, `y_test`) has been completely untouched during training and tuning.
- We use the `best_estimator_` from the search to generate predictions.

Generating Predictions

- `y_pred = best_model.predict(X_test)` (Class labels)
- `y_prob = best_model.predict_proba(X_test)[:, 1]` (Probabilities for ROC/PR curves)

This simulates real-world deployment on unseen patient data.

Comprehensive Evaluation Metrics

- **Accuracy:** Overall correctness (can be misleading if the dataset is imbalanced).
- **Precision:** Out of all predicted positives, how many are actual positives? (Minimizes False Positives).
- **Recall (Sensitivity):** Out of all actual positives, how many were predicted? (Minimizes False Negatives - crucial for healthcare).
- **F1-Score:** Harmonic mean of Precision and Recall. Balances both concerns.

Understanding ROC-AUC and PR-AUC

ROC-AUC (Receiver Operating Characteristic)

- Plots True Positive Rate vs. False Positive Rate across all thresholds.
- Measures the model's overall ability to distinguish between classes.

PR-AUC (Precision-Recall Curve)

- Plots Precision vs. Recall.
- Highly informative for heavily imbalanced datasets, as it focuses strictly on the minority (positive) class.

Generating and Analyzing Confusion Matrices

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot(cmap='Blues')
plt.title(f'Confusion Matrix - {model_name}')
plt.show()
```

Visualizing the confusion matrix helps quickly identify if the model is biased towards False Positives or False Negatives.

Side-by-Side Model Comparison

Model	Accuracy	Recall	Precision	F1	ROC-AUC
Log. Reg.	0.78	0.82	0.75	0.78	0.84
SVM	0.79	0.80	0.78	0.79	0.85
Random Forest	0.82	0.85	0.80	0.82	0.88
XGBoost	0.84	0.88	0.82	0.85	0.91

Initial Findings

XGBoost and Random Forest emerge as the top-performing models, offering the highest Recall and ROC-AUC, making them ideal candidates for clinical deployment.

Summary & Best Practices

1. **Pipelines:** Essential for preventing data leakage and ensuring reproducible preprocessing.
2. **Tuning:** `RandomizedSearchCV` efficiently finds optimal hyperparameters while saving computational time.
3. **Evaluation:** Look beyond accuracy; prioritize Recall and ROC-AUC in clinical settings to minimize missed diagnoses.
4. **Next Steps:** We will take the top-performing models and apply advanced techniques like threshold tuning and ensemble stacking.

Questions?

Lumumba W. Victor
Trainer — Data Analyst — ML Expert